

```
// =====

// ESP8266 — LoRa Transmitter (Rover Side) + GPS v5

//

// Pico W UART1 GP4(TX) → ESP8266 GPIO3(RX) @ 115200
// Pico W UART1 GP5(RX) ← ESP8266 GPIO1(TX) @ 115200

//

// HOW IT WORKS:

// 1. Pico W sends a framed message over UART
// e.g. <MSG|R1|HUMAN_DETECTED|LAT:12.971600|LON:77.594600>

// 2. ESP8266 reads it character by character until \n

// 3. Validates the frame (must start < end > have | fields)

// 4. GPS footer is appended before closing >

// Format: |lat,lon,HH:MM:SS,DD/MM/YY

// 5. Transmits over LoRa with up to 3 retries

// 6. All debug prints go to Serial (USB monitor)

//

// LoRa SPI : SCK=14(D5) MISO=12(D6) MOSI=13(D7)

// NSS=15(D8) RST=16(D0) DIO0=5(D1)

//

// GPS UART : RX=D3(GPIO0) ← GPS TX

// TX=D4(GPIO2) → GPS RX @ 9600

//

// PIN CONFLICT RESOLUTION:

// Previous GPS used D6/D7 which clashed with LoRa MISO/MOSI.

// Moved GPS SoftwareSerial to D3/D4 — completely free pins.

//

// Libraries: sandeepmistry/arduino-LoRa

// mikalhart/TinyGPSPlus
```

```

// =====

#include <Arduino.h>

#include <SPI.h>

#include <LoRa.h>

#include <TinyGPS++.h>

#include <SoftwareSerial.h>

// -----

// LoRa pins — UNCHANGED

// -----

#define LORA_SCK 14 // D5

#define LORA_MISO 12 // D6

#define LORA_MOSI 13 // D7

#define LORA_NSS 15 // D8

#define LORA_RST 16 // D0

#define LORA_DIO0 5 // D1

// -----

// LoRa settings — MUST match RX node exactly — UNCHANGED

// -----

#define LORA_FREQ 433E6

#define LORA_SF 7

#define LORA_BW 125E3

#define LORA_CR 5 // coding rate 4/5

// -----

// UART / buffer settings — UNCHANGED

```

```

// -----

#define UART_BAUD 115200

#define MAX_BUF 200

#define MAX_RETRIES 3

#define RETRY_DELAY_MS 300

#define REINIT_MS 5000

// -----

// GPS pins — D3/D4 chosen to avoid SPI bus conflict
// D6(MISO) and D7(MOSI) are used by LoRa SPI,
// so GPS moved to D3(GPIO0) and D4(GPIO2)
// -----

#define GPS_RX_PIN 0 // D3 — connects to GPS TX
#define GPS_TX_PIN 2 // D4 — connects to GPS RX
#define GPS_BAUD 9600

TinyGPSPlus gps;
SoftwareSerial gpsSerial(GPS_RX_PIN, GPS_TX_PIN);

// -----

// State — UNCHANGED
// -----

String rxBuf = "";

bool msgReady = false;

bool loraOK = false;

uint32_t lastReinit = 0;

uint32_t txOK = 0;

uint32_t txFail = 0;

```

```

uint32_t rxFromPico = 0;

// -----
// initLoRa() — UNCHANGED
// -----

bool initLoRa() {
    LoRa.end();
    delay(100);
    SPI.begin();
    LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

    if (!LoRa.begin(LORA_FREQ)) {
        Serial.println("[TX] ERROR: LoRa.begin() failed");
        return false;
    }

    LoRa.setSpreadingFactor(LORA_SF);
    LoRa.setSignalBandwidth(LORA_BW);
    LoRa.setCodingRate4(LORA_CR);
    LoRa.enableCrc();

    Serial.println("[TX] LoRa OK — 433MHz SF7 BW125 CR4/5 CRC ON");
    return true;
}

// -----
// isValid() — UNCHANGED
// -----

```

```

bool isValid(const String& msg) {
    if (msg.length() == 0) {
        Serial.println("[TX] SKIP: empty message");
        return false;
    }
    if (msg.length() > MAX_BUF) {
        Serial.print("[TX] SKIP: too long (");
        Serial.print(msg.length());
        Serial.println(" bytes)");
        return false;
    }
    if (!msg.startsWith("<")) {
        Serial.print("[TX] SKIP: no < start → ");
        Serial.println(msg);
        return false;
    }
    if (!msg.endsWith(">")) {
        Serial.print("[TX] SKIP: no > end → ");
        Serial.println(msg);
        return false;
    }
    if (msg.indexOf('|') == -1) {
        Serial.print("[TX] SKIP: no | field → ");
        Serial.println(msg);
        return false;
    }
    return true;
}

```

```

// -----
// transmit() — UNCHANGED
// -----

bool transmit(const String& msg) {
    for (int attempt = 1; attempt <= MAX_RETRIES; attempt++) {
        Serial.print("[TX] Attempt ");
        Serial.print(attempt);
        Serial.print("/");
        Serial.print(MAX_RETRIES);
        Serial.print(" → ");
        Serial.println(msg);

        if (!LoRa.beginPacket()) {
            Serial.println("[TX] beginPacket() failed");
            delay(RETRY_DELAY_MS);
            continue;
        }

        int written = LoRa.print(msg);
        if (written == 0) {
            Serial.println("[TX] print() wrote 0 bytes");
            LoRa.endPacket();
            delay(RETRY_DELAY_MS);
            continue;
        }

        if (LoRa.endPacket(false) == 1) {

```

```
    txOK++;

    Serial.print("[TX] Sent OK | Total OK: ");

    Serial.print(txOK);

    Serial.print(" Fail: ");

    Serial.println(txFail);

    return true;

} else {

    Serial.println("[TX] endPacket() failed");

    delay(RETRY_DELAY_MS);

}

}
```

```
txFail++;

loraOK = false;

Serial.print("[TX] FAILED after ");

Serial.print(MAX_RETRIES);

Serial.print(" attempts | Total OK: ");

Serial.print(txOK);

Serial.print(" Fail: ");

Serial.println(txFail);

return false;

}
```

```
// -----

// readUART() — UNCHANGED

// -----

void readUART() {

    while (Serial.available()) {
```

```

char c = (char)Serial.read();

if (c == '\n') {
    rxBuf.trim();
    if (rxBuf.length() > 0) {
        msgReady = true;
        rxFromPico++;
    }
    return;
}

if (c == '\r') continue;

if (rxBuf.length() < MAX_BUF) {
    rxBuf += c;
} else {
    Serial.println("[TX] WARN: buffer overflow — flushed");
    rxBuf = "";
    msgReady = false;
}
}

// -----
// buildGpsFooter() — NEW
// Appended to frame before closing '>'
// Output: |lat,lon,HH:MM:SS,DD/MM/YY
// No fix: |--,--,:--:--/--/--

```



```
// -----
```

```
String buildGpsFooter() {
```

```
    String footer = "|";
```

```
    // Latitude, Longitude
```

```
    if (gps.location.isValid()) {
```

```
        footer += String(gps.location.lat(), 6);
```

```
        footer += ",";
```

```
        footer += String(gps.location.lng(), 6);
```

```
    } else {
```

```
        footer += "--,--";
```

```
    }
```

```
    footer += ",";
```

```
    // Time HH:MM:SS (GPS time is UTC)
```

```
    if (gps.time.isValid()) {
```

```
        char t[9];
```

```
        sprintf(t, "%02d:%02d:%02d",
```

```
            gps.time.hour(),
```

```
            gps.time.minute(),
```

```
            gps.time.second());
```

```
        footer += t;
```

```
    } else {
```

```
        footer += "--:--:--";
```

```
    }
```

```
    footer += ",";
```

```

// Date DD/MM/YY
if (gps.date.isValid()) {
    char d[9];
    sprintf(d, "%02d/%02d/%02d",
            gps.date.day(),
            gps.date.month(),
            gps.date.year() % 100);
    footer += d;
} else {
    footer += "--/--/--";
}

return footer;

// e.g. |12.971600,77.594600,09:15:42,07/06/25
}

// -----
// setup()
// -----

void setup() {
    Serial.begin(UART_BAUD);
    delay(500);

    Serial.println();
    Serial.println("=====");
    Serial.println(" ESP8266 LoRa Transmitter + GPS — Rover");
    Serial.println(" Pico GP4(TX) → GPIO3(RX) @ 115200");

```

```

Serial.println(" GPS RX=D3(GPIO0) TX=D4(GPIO2) @ 9600");

Serial.println("=====");

// GPS SoftwareSerial on D3/D4
gpsSerial.begin(GPS_BAUD);
Serial.println("[TX] GPS serial started — D3(RX) / D4(TX)");

loraOK = initLoRa();

if (!loraOK) {
  Serial.println("[TX] LoRa failed — will retry every 5s");
} else {
  Serial.println("[TX] Waiting for messages from Pico ...");
  Serial.println();
}
}

// -----
// loop()
// -----

void loop() {

  // — Feed GPS parser — must run every loop tick —————
  while (gpsSerial.available()) {
    gps.encode(gpsSerial.read());
  }

  // — Auto-reinit LoRa if it failed —————

```

```

if (!loraOK) {
    if (millis() - lastReinit >= REINIT_MS) {
        lastReinit = millis();
        Serial.println("[TX] Retrying LoRa init ...");
        loraOK = initLoRa();
        if (loraOK) Serial.println("[TX] LoRa back up — ready");
    }
}

// — Read from Pico W via UART —————
readUART();

// — Process received message —————
if (msgReady) {
    Serial.println("-----");
    Serial.print("[TX] From Pico #");
    Serial.print(rxFromPico);
    Serial.print(" → ");
    Serial.println(rxBuf);

    if (!isValid(rxBuf)) {
        // reason printed inside isValid()

    } else if (!loraOK) {
        Serial.println("[TX] LoRa not ready — message dropped");
        txFail++;
    } else {

```

```
// Append GPS footer: strip '>', add footer, restore '>'
String outMsg = rxBuf;
outMsg.remove(outMsg.length() - 1); // remove '>'
outMsg += buildGpsFooter();      // |lat,lon,time,date
outMsg += ">";                    // restore '>'

Serial.print("[TX] Final frame → ");
Serial.println(outMsg);
transmit(outMsg);
}

Serial.println("-----");
Serial.println();

rxBuf = "";
msgReady = false;
}
}
```